

Crakmes

Crackme: EasiestEver by BadEngineer

Challenge Overview

Challenge: <https://crackmes.one/crackme/697fd52a16739b40dcb5dabd>

This is a simple Windows PE32+ binary that implements a password-checking mechanism. The challenge is to find a valid password that passes the `PasswordCheck` function and unlocks the secret message.

Binary Analysis

File Information

- **File Type:** Windows PE32+ executable (64-bit)
- **Compiler:** MinGW-w64 (GCC 15.2.0)
- **Architecture:** x86-64

Main Function

```
int __cdecl main(int _Argc, char **_Argv, char **_Env)
{
    undefined8 uVar1;
    char local_18[16];

    __main();
    __mingw_printf("Please enter the password : ");
    __mingw_scanf("%s", local_18);
    while (true) {
        uVar1 = PasswordCheck(local_18);
        if ((int)uVar1 != 0) break;
        __mingw_printf("Wrong Password, Try again : ");
        __mingw_scanf("%s", local_18);
    }
}
```

```

    }
    __mingw_printf("Congrats, You deserve this!");
    ShellExecuteA((HWND)0x0, (LPCSTR)0x0, "<https://www.youtube.com/
watch?v=dQw4w9WgXcQ>", (LPCSTR)0x0, (LPCSTR)0x0, 5);
    return 0;
}

```

The main function prompts the user for a password, checks it using `PasswordCheck`, and repeats until a valid password is entered. If successful, it shows a congratulatory message and opens a YouTube link.

PasswordCheck Function Analysis

Decompiled Code

```

undefined8 PasswordCheck(char *param_1)
{
    char cVar1;
    size_t sVar2;
    undefined8 uVar3;
    int local_6c;
    longlong local_68;
    longlong local_60;
    longlong local_58;
    longlong local_50;
    longlong local_48;
    longlong local_40;
    longlong local_38;
    longlong local_30;
    longlong local_28;
    longlong local_20;

    // Initialize all counters to 0
    local_20 = 0;
    local_28 = 0;
    local_30 = 0;
    local_38 = 0;
    local_40 = 0;
    local_48 = 0;
    local_50 = 0;

```

```

local_58 = 0;
local_60 = 0;
local_68 = 0;

sVar2 = strlen(param_1);
if (sVar2 < 0x11) { // Password must be <= 16 characters
    local_6c = 0;
    while (true) {
        sVar2 = strlen(param_1);
        if (sVar2 <= (ulonglong)(longlong)local_6c) break;
        cVar1 = param_1[local_6c];

        // Character range checking
        if ((cVar1 < '0') || ('4' < cVar1)) {
            if ((cVar1 < 'H') || ('N' < cVar1)) {
                if ((cVar1 < 't') || ('y' < cVar1)) {
                    if ((cVar1 < 'a') || ('f' < cVar1)) {
                        if ((cVar1 < '!') || ('&' < cVar1)) {
                            if ((cVar1 < ';' ) || ('?' < cVar1))
                                if ((cVar1 < 'j') || ('m' < cVar1))
                                    if ((cVar1 < 'z') || ('}' < cVar1))
                                        if ((cVar1 < 'o') || ('s' < cVar1))
                                            if (('[' < cVar1) && (cVar1 < 'a'))
                                                local_68 = local_68 + 1;
                                }
                            }
                        else {
                            local_60 = local_60 + 1;
                        }
                    }
                else {
                    local_58 = local_58 + 1;
                }
            }
        }
    }
}

```

```

        }
        else {
            local_50 = local_50 + 1;
        }
    }
    else {
        local_48 = local_48 + 1;
    }
}
else {
    local_40 = local_40 + 1;
}
}
else {
    local_38 = local_38 + 1;
}
}
else {
    local_30 = local_30 + 1;
}
}
else {
    local_28 = local_28 + 1;
}
}
else {
    local_20 = local_20 + 1;
}
local_6c = local_6c + 1;
}

// Check all counters are non-zero
if (((((((local_20 == 0) || (local_28 == 0)) || (local_30 ==
0)) ||
        ((local_38 == 0 || (local_40 == 0)))) ||
        ((local_48 == 0 || ((local_50 == 0 || (local_58 ==
0)))))) || (local_60 == 0)) ||
    (local_68 == 0)) {
    uVar3 = 0;
}

```

```

        else {
            uVar3 = 1;
        }
    }
    else {
        uVar3 = 0; // Password too long
    }
    return uVar3;
}

```

Exact Character Ranges (from assembly)

The `PasswordCheck` function checks for characters in **exactly 10 specific ranges**. Each range must contain at least one character for the password to be valid.

[Keygen.py](#)

```

#!/usr/bin/env python3

# Exact character ranges from assembly
ranges = [
    (0x2F, 0x34), # '/' to '4'
    (0x47, 0x4E), # 'G' to 'N'
    (0x73, 0x79), # 's' to 'y'
    (0x60, 0x66), # '`' to 'f'
    (0x20, 0x26), # ' ' to '&'
    (0x3A, 0x3F), # ':' to '?'
    (0x69, 0x6D), # 'i' to 'm'
    (0x79, 0x7D), # 'y' to '}'
    (0x6E, 0x73), # 'n' to 's'
    (0x5B, 0x60), # '[' to '`'
]

def get_chars_in_range(start, end):
    """Get all characters in a range"""
    return [chr(i) for i in range(start, end + 1)]

# Get characters for each range
range_chars = [get_chars_in_range(start, end) for start, end in rang

```

```

es]

print("Character ranges:")
for i, chars in enumerate(range_chars):
    print(f"Range {i+1}: {''.join(chars)}")

# Create a systematic password with one from each range
# We need 16 chars total, so 6 extra chars that don't interfere
base_chars = []
for chars in range_chars:
    base_chars.append(chars[0]) # Take first char from each range

# Fill remaining 6 positions with safe characters (from range 1)
remaining = 16 - len(base_chars)
for i in range(remaining):
    base_chars.append(range_chars[0][0])

password = ''.join(base_chars)
print(f"\nSystematic password: {password}")
print(f"Length: {len(password)}")

# Verify all ranges are represented
print("\nVerification:")
for i, (start, end) in enumerate(ranges):
    has_char = any(start <= ord(c) <= end for c in password)
    print(f"Range {i+1} ({chr(start)}-{chr(end)}): {'✓' if has_char
    else 'x'}")

# Generate a few systematic variations
print("\nSystematic variations:")
for i in range(min(5, len(range_chars[0]))):
    chars = []
    for j, range_ch in enumerate(range_chars):
        chars.append(range_ch[i % len(range_ch)])
    # Fill remaining
    while len(chars) < 16:
        chars.append(range_chars[0][0])
    print(''.join(chars))

```

```
/Gs` :iyn[/////
0Hta!;jzo\/////
1lub"<k{p]/////
2Jvc#=#|q^/////
3Kwd$>m}r_/////
```

Key Requirements

1. **Length:** Exactly 16 characters (must be ≤ 16 , but optimal is exactly 16)
2. **Character Diversity:** Must contain at least one character from **each of the 10 ranges**
3. **No Null Bytes:** Standard C string (null-terminated)

Solution

Valid Password Generation

To solve this challenge, we need to generate passwords that include at least one character from each of the 10 ranges, and are exactly 16 characters long.

Keygen Implementation

I created a Python keygen ([keygen.py](#)) that:

1. Generates valid passwords following all rules
2. Excludes problematic characters (quotes for easier handling)
3. Verifies password validity
4. Provides interactive and command-line interfaces

Example Valid Passwords

Here are 10 valid passwords generated by the keygen:

```
%c};[xara|kNH\m3
^cpyx%2a;k;n{3lJ
o{x$n:x:bGL`2mK[
kb`/yv$p#Hwp|:nb
fys#|ii<qKv4Lq_=
M`lz}aiNn];lt!`>
usz%;No`\\j3wd2[$
sx\\3Hz$[f4oIik?y
```

```
nxG3]\m!{r^z`<zj  
/_$xv4<n;ymv\cNd
```

Manual Verification

One systematic password that works is:

```
0Hta!;jzo\\/////
```

This password contains exactly one character from each range and is 16 characters long.

Conclusion

This is a straightforward crackme challenge that tests basic reverse engineering skills. The password validation logic is simple but requires understanding of character range checks and counting mechanisms.

Lessons Learned

1. **Assembly Analysis:** Understanding the exact character ranges from assembly was crucial for solving this challenge.
2. **Systematic Approach:** Creating a script to systemically generate valid passwords simplifies the process.
3. **Edge Cases:** Being aware of problematic characters (like quotes) makes the solution more practical.

This challenge demonstrates how reverse engineering can be applied to simple binary protections to bypass security checks.